

AFRILANG Stdlib

std/c/parsexml

Français. Bibliothèque AFRILANG 'std/c/parsexml' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : xmlLen, xmlUpper, xmlLower, xmlTrim, xmlSplit, xmlJoin, xmlReplace.

'import "std/c/parsexml"' · 'use parsexml'

- 'xmlLen(s text) → number' - 'xmlUpper(s text) → text' - 'xmlLower(s text) → text' - 'xmlTrim(s text) → text' - 'xmlSplit(s text, delim text) → list text' - 'xmlJoin(parts list text, delim text) → text' - 'xmlReplace(s text, from text, to text) → text'

English. AFRILANG library 'std/c/parsexml' (tier complex, category complex). Exports 7 function(s): xmlLen, xmlUpper, xmlLower, xmlTrim, xmlSplit, xmlJoin, xmlReplace.

'import "std/c/parsexml"' · 'use parsexml'

- 'xmlLen(s text) → number' - 'xmlUpper(s text) → text' - 'xmlLower(s text) → text' - 'xmlTrim(s text) → text' - 'xmlSplit(s text, delim text) → list text' - 'xmlJoin(parts list text, delim text) → text' - 'xmlReplace(s text, from text, to text) → text'

Catégorie / Category Modules complex (c/)

Niveau / Tier complex

Fonctions / Functions 7

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/parsexml' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : xmlLen, xmlUpper, xmlLower, xmlTrim, xmlSplit, xmlJoin, xmlReplace.

```
'import "std/c/parsexml"' · 'use parsexml'
```

```
- `xmlLen(s text) → number`
- `xmlUpper(s text) → text`
- `xmlLower(s text) → text`
- `xmlTrim(s text) → text`
- `xmlSplit(s text, delim text) → list text`
- `xmlJoin(parts list text, delim text) → text`
- `xmlReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/parsexml"
use parsexml
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- xmlLen(s text) → number•
xmlUpper(s text) → text•
xmlLower(s text) → text•
xmlTrim(s text) → text•
xmlSplit(s text, delim text) → list•
xmlJoin(parts list text, delim text) → text•
xmlReplace(s text, from text, to text) → text

4. Exemples d'utilisation

```
import "std/c/parsexml"
use parsexml
```

```
create result = xmlLen(/* args */)
say result
```

```
create result = xmlUpper(/* args */)
say result
```

```
create result = xmlLower(/* args */)
say result
```

```
create result = xmlTrim(/* args */)
say result
```

```
create result = xmlSplit(/* args */)
say result
```

```
create result = xmlJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/parsexml"`` en tête de fichier.
- Lancez ``afrilang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module parsexml

```
function xmlLen(s text) returns number
end
```

```
function xmlUpper(s text) returns text
end
```

```
function xmlLower(s text) returns text
end
```

```
function xmlTrim(s text) returns text
end
```

```
function xmlSplit(s text, delim text) returns list text
end
```

```
function xmlJoin(parts list text, delim text) returns text
end
```

```
function xmlReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/parsexml' (tier complex, category complex). Exports 7 function(s): xmlLen, xmlUpper, xmlLower, xmlTrim, xmlSplit, xmlJoin, xmlReplace.

```
'import "std/c/parsexml"' · 'use parsexml'
```

```
- `xmlLen(s text) → number`  
- `xmlUpper(s text) → text`  
- `xmlLower(s text) → text`  
- `xmlTrim(s text) → text`  
- `xmlSplit(s text, delim text) → list text`  
- `xmlJoin(parts list text, delim text) → text`  
- `xmlReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/parsexml"  
use parsexml
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- xmlLen(s text) → number•
xmlUpper(s text) → text•
xmlLower(s text) → text•
xmlTrim(s text) → text•
xmlSplit(s text, delim text) → list•
xmlJoin(parts list text, delim text) → text•
xmlReplace(s text, from text, to text) → text

4. Usage examples

```
import "std/c/parsexml"  
use parsexml
```

```
create result = xmlLen(/* args */)   
say result
```

```
create result = xmlUpper(/* args */)   
say result
```

```
create result = xmlLower(/* args */)   
say result
```

```
create result = xmlTrim(/* args */)   
say result
```

```
create result = xmlSplit(/* args */)   
say result
```

```
create result = xmlJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/parsexml"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module parsexml

```
function xmlLen(s text) returns number
end
```

```
function xmlUpper(s text) returns text
end
```

```
function xmlLower(s text) returns text
end
```

```
function xmlTrim(s text) returns text
end
```

```
function xmlSplit(s text, delim text) returns list text
end
```

```
function xmlJoin(parts list text, delim text) returns text
end
```

```
function xmlReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/parsexml"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/parsexml/>

Source (extrait)

```
module parsexml

function xmlLen(s text) returns number
end

function xmlUpper(s text) returns text
```

```
end

function xmlLower(s text) returns text
end

function xmlTrim(s text) returns text
end

function xmlSplit(s text, delim text) returns list text
end

function xmlJoin(parts list text, delim text) returns text
end

function xmlReplace(s text, from text, to text) returns text
end
end
```